

Wydział Informatyki

**PROGRAMOWANIE APLIKACJI INTERNETOWYCH W
TECHNOLOGII JAVA**

Temat projektu: **Noteally - aplikacja do tworzenia notatek (na
podstawie wymagań StoreEverything)**

Wykonujący: **Izabela Klis**

Jakub Komarewski

Mateusz Mogielnicki

Studia dzienne

Kierunek: **Informatyka**

Rok: **II**

Semestr: **IV**

Grupa zajęciowa: **PS6**

Prowadzący ćwiczenie: **dr inż. Urszula Kuźelewska**

Data wykonania ćwiczenia:

16.06.2023r.

OPIS WSZYSTKICH PUNKTOWANYCH FUNKCJONALNOŚCI

1. Role Użytkowników

- Admin - zarządza użytkownikami,
- Limited user - użytkownik z ograniczonymi prawami, jest zarejestrowany, może przeglądać udostępnione linki, ale nie może tworzyć informacji,
- Full user - użytkownik z pełnymi prawami, może tworzyć informacje i udostępniać je innym, przeglądać w oddzielnym widoku informacje udostępnione dla niego
- Użytkownik niezalogowany - ma dostęp tylko do strony początkowej i strony rejestracji

2. Szczegółowe funkcjonalności, full user (19p)

- dodanie/edycja/usunięcie przez siebie zebranych informacji - 5p.
- Walidacja formularza - 1p.
- Edycja na danych bieżących - 1p.
- Dodanie nowej kategorii - 1p.
- Wyświetlenie udostępnionych przez innych informacji - 2p.
- udostępnienie: ze wskazaniem na konkretnego użytkownika lub w linku - 1p.
- Wyświetlanie "swoich" informacji: sortowanie w obu kierunkach (data,kategoria, alfabetycznie) - 2p.
- Zapamiętanie kierunków i kryteriów sortowania - 1p.
- Filtrowanie według daty (od aktualnej) i kategorii (od najbardziej popularnej) - 2p.
- Logowanie - 1p.
- Zapis do bazy danych dopiero przy wylogowaniu/wygaśnięciu sesji - 2p.

3. Szczegółowe Funkcjonalności, Niezalogowany (4p)

- Rejestracja - 1p.
- Walidacja formularza - 1p.
- Strona powitalna - 1p.
- Wyświetlenie informacji z udostępnionego linku - 1p.

4. Szczegółowe Funkcjonalności, admin (2p)

- Wyśw. listy użytkowników - 1p.
- Zarządzanie rolami - 1p.

5. Elementy Techniczne (25p.)

- Kontrolery - 2p.
- Baza danych (co najmniej 2 tabele z relacją) - 5p.
- Widoki: formularze z walidacją (3 różne elementy),
- 5 różnych znaczników Thymeleafa - 3p.+2p.
- Sesja - 2p.
- Ciasteczka - 1p.
- Usługa REST (do uwierzytelniania użytkowników) - 3p.
- Wygląd - (WCAG 2.1)
- Spring Security - 5p. (z bazą danych), 3p. (w pamięci)

IMPLEMENTACJA WYMAGAŃ Z PUNKTU 1 (ROLE)

(Opis szczegółowych funkcjonalności znajduje się w późniejszych fragmentach)

Admin - Admin ze swojego panelu ma możliwość edytowania wszystkich danych użytkowników. Może również usunąć użytkownika z serwisu.

Użytkownik „Limited” - użytkownik z ograniczonymi prawami nie może tworzyć żadnych notatek. Można mu natomiast udostępniać notatki, jeżeli posiada się prawa „Full User” lub „Admin”.

Użytkownik „Full User” - Użytkownik z pełnymi prawami może tworzyć, usuwać i edytować katalogi (foldery z notatkami). W tych katalogach może on tworzyć, usuwać i edytować notatki. Może również udostępniać notatki.

Użytkownik niezalogowany - Dla osoby niezalogowanej jedyną dostępną funkcjonalnością jest możliwość zalogowania się lub utworzenia nowego konta w serwisie.

IMPLEMENTACJA WYMAGAŃ Z PUNKTU 2 (FULL USER)

Dodanie/edycja/usunięcie przez siebie zebranych informacji - 5p.

Dodawanie/edycja i usuwanie notatek możliwe jest dzięki odpowiednim przekierowaniom z pliku **notes.html**

```
39 <a th:href="@{/catalogs/ + ${catalog.getId()} + '/createNote'}"
92 <a th:href="@{/catalogs/ + ${catalog.getId()} + '/deleteNote/ + ${note.getId()}"
93 ><i class="fa-solid fa-xmark"></i>
94 ></a>
95 <a th:href="@{/catalogs/ + ${catalog.getId()} + '/editNote/ + ${note.getId()}"
96 ><i class="fa-regular fa-pen-to-square"></i>
97 ></a>
```

Przekierowania wyłapywane są przez kontroler **NotesController.java**, który wykonuje odpowiednie operacje.

Tworzenie:

```
309 @GetMapping("/{createNote}")
310 public String redirectCreate(Model model, @PathVariable("catalogId") Integer catalogId, HttpSession session) {
311     Optional<Catalog> catalog = catalogService.getCatalogById(catalogId);
312     Integer userId = (Integer) session.getAttribute("userId");
313     if (catalog.isPresent() && catalog.get().getUser().getId().equals(userId) && !(catalog.get().getName().equals("shared"))) {
314         Note note = new Note();
315         model.addAttribute("note", note);
316         model.addAttribute("catalog", catalog.get());
317         return "createNote";
318     }
319     return "redirect:/catalogs";
320 }
321
322 @PostMapping("/{createNote}")
323 public String addNote(@Valid @ModelAttribute("note") Note note, BindingResult bindingResult, Model model, @PathVariable("catalogId") Integer catalogId) {
324     if (bindingResult.hasErrors()) {
325         model.addAttribute("errors", bindingResult);
326         Optional<Catalog> catalog = catalogService.getCatalogById(catalogId);
327         model.addAttribute("catalog", catalog.get());
328         return "createNote";
329     }
330 }
```

Usuwanie:

```
335 @GetMapping("/{deleteNote/{noteId}}")
336 public String delete(Model model, @PathVariable("noteId") Integer noteId, @PathVariable("catalogId") Integer catalogId, HttpSession session) {
337     Integer userId = (Integer) session.getAttribute("userId");
338     Optional<Catalog> catalog = catalogService.getCatalogById(catalogId);
339     if (catalog.isPresent() && catalog.get().getUser().getId().equals(userId) && !(catalog.get().getName().equals("shared"))) {
340         noteService.deleteNoteById(noteId);
341         return "redirect:/catalogs/" + catalogId;
342     }
343     return "redirect:/catalogs";
344 }
```

Edycja:

```
345 @GetMapping("/{editNote/{noteId}}")
346 public String redirectEdit(Model model, @PathVariable("noteId") Integer noteId, @PathVariable("catalogId") Integer catalogId, HttpSession session) {
347     Optional<Catalog> catalog = catalogService.getCatalogById(catalogId);
348     Integer userId = (Integer) session.getAttribute("userId");
349     if (catalog.isPresent() && catalog.get().getUser().getId().equals(userId) && !(catalog.get().getName().equals("shared"))) {
350         Optional<Note> note = noteService.getNoteById(noteId);
351         model.addAttribute("note", note.get());
352         model.addAttribute("catalog", catalog.get());
353         return "editNote";
354     }
355     return "redirect:/catalogs";
356 }
357
358 @PostMapping("/{editNote/{noteId}}")
359 public String editNote(@Valid @ModelAttribute("note") Note note, BindingResult bindingResult, @PathVariable("noteId") Integer noteId, @PathVariable("catalogId") Integer catalogId) {
360     if (bindingResult.hasErrors()) {
361         note.setId(noteId);
362         model.addAttribute("note", note);
363         model.addAttribute("errors", bindingResult);
364         Optional<Catalog> catalog = catalogService.getCatalogById(catalogId);
365         model.addAttribute("catalog", catalog.get());
366         return "editNote";
367     }
368 }
```

Metody wyłapujące żądania GET przekierowują odpowiednio na widoki **createNote.html** i **editNote.html**, które po wypełnieniu formularzy przekierowują na odpowiednie linki z metodami POST.

Walidacja formularza - 1p.

Walidacja została zaimplementowana w klasach, a błędy walidacji są wyświetlane w odpowiednich widokach. Walidacja na przykładzie rejestracji:

Fragment klasy danych **User.java**:

```
31 @Basic
32 @NotBlank(message = "Must not be empty.")
33 @Column(name = "login", nullable = false)
34 @Size(min = 3, max = 20, message = "Size must be between 3 and 20.")
35 @Pattern(regexp = "^[a-z]+$", message = "Login must contain only small letters.")
36 private String login;
37
38 1 usage
39 @Basic
40 @NotBlank(message = "Password is required.")
41 @Size(min = 6, max = 20, message = "Must be longer than 5 letters.")
42 @Column(name = "password", nullable = false)
43 private String password;
44
45 1 usage
46 @Basic
47 @NotBlank(message = "Must not be empty.")
48 @Column(name = "name", nullable = false)
49 @Size(min = 3, max = 20, message = "Size must be between 3 and 20")
50 @Pattern(regexp = "^[A-Z][a-z]*$", message = "Name must start with capital letter and contain only letters.")
51 private String name;
```

Fragment widoku rejestracji **register.html**:

```
14 <form th:action="@{/register}" th:method="post" th:object="${user}">
15   <h2 class="note">Registration<span class="ally">Form</span></h2>
16   <div class="input-box">
17     <span><i class="fa-sharp fa-solid fa-user"></i></span>
18     <input type="text" id="username" name="username" th:field="*{login}" th:value="${username}"/>
19     <label for="username">Username<div th:if="${#fields.hasErrors('login')} || usernameTaken!=null" class="error-label-name">
20       <span th:if="${#fields.hasErrors('login')}" th:errors="*{login}"></span>
21       <span th:if="${usernameTaken!=null}" th:text="${usernameTaken}"></span></div></label>
22   </div>
23   <div class="input-box">
24     <span><i class="fa-sharp fa-solid fa-signature"></i></span>
25     <input type="text" id="name" name="name" th:field="*{name}" th:value="${name}"/>
26     <label for="name">Name<div th:if="${#fields.hasErrors('name')} || polishNameError!=null" class="error-label-name">
27       <span th:if="${#fields.hasErrors('name')}" th:errors="*{name}"></span>
28       <span th:if="${polishNameError!=null}" th:text="${polishNameError}"></span></div></label>
29   </div>
30   <div class="input-box">
31     <span><i class="fa-sharp fa-solid fa-signature"></i></span>
32     <input type="text" id="surname" name="surname" th:field="*{surname}" th:value="${surname}"/>
33     <label for="surname">Surname<div class="error-label-name" th:if="${#fields.hasErrors('surname')}" th:errors="*{surname}"></div></label>
```

Widok w aplikacji przy błędzie walidacji formularza rejestracji:

Size must be between 3 and 20.

Name must start with capital letter and contain only letters. Name must be Polish.

Name must start with capital letter and contain only letters. Must not be empty. Size must be between 3 and 50.

You must be 18 or older.

Must be longer than 5 letters.

Passwords are different

RegistrationForm

Username

ab

Name

aaaaaaaaaaa2

Surname

Age

13

Password

Confirm Password

Register

Edycja na danych bieżących - 1p.

Fragment widoku **editNote.html** odpowiadający za wstawienie bieżących danych przy otwarciu formularza (znaczniki **th:field** importują dane z edytowanej notatki):

```

67 <form
68   th:action="@{/catalogs/' + ${catalog.id} + '/editNote/' + ${note.getId()}}"
69   th:method="POST"
70   th:object="${note}">
71   <h2 class="note">Edit<span class="ally">Note</span></h2>
72   <div class="input-box">
73     <input type="text" id="title" names="title" th:field="${title}" />
74     <label for="title">Title<div class="error-label-name" th:if="${#fields.hasErrors('title')}" th:errors="${title}"></div></label>
75   </div>
76   <div class="input-box">
77     <textarea
78       cols="30"
79       rows="10"
80       type="text"
81       id="content"
82       name="content"
83       th:field="${content}"></textarea>
84     <label class="content" for="content">Content<div class="error-label-name" th:if="${#fields.hasErrors('content')}" th:errors="${content}"></div></label>
85   </div>
86   <div class="input-box">
87     <input type="text" id="link" name="link" th:field="${link}" />
88     <label for="link">link</label>

```

Dodanie nowej kategorii - 1p.

Zamiast kategorii notatek, użytkownicy mogą tworzyć katalogi (foldery), w których trzymają swoje notatki. Tytuły katalogów mogą pełnić rolę kategorii, w zależności od potrzeb użytkownika.

Dodawanie/edycja i usuwanie katalogów możliwe jest dzięki odpowiednim przekierowaniom z widoku **catalogs.html**:

```
27 <div class="add-note anim" style="...">
28   <a th:href="@{'/catalogs/' + 'createCatalog'}"><i class="fa-solid fa-folder-plus"></i></a>
29 </div>

54 <div th:if="${catalog.getName() != 'default' && catalog.getName() != 'shared'}">
55   <a th:href="@{'/catalogs/' + 'deleteCatalog/' + ${catalog.getId()}}">
56     <i class="fa-solid fa-xmark"></i>
57   </a>
58
59   <a th:href="@{'/catalogs/' + 'editCatalog/' + ${catalog.getId()}}">
60     <i class="fa-regular fa-pen-to-square"></i>
61   </a>
```

Przekierowania wyłapywane są przez kontroler **CatalogsController.java**, który wykonuje odpowiednie operacje.

Dodawanie:

```
118 @GetMapping("/{createCatalog}")
119 public String redirectCreate(Model model, HttpSession session){
120     Catalog catalog = new Catalog();
121     Optional<User> user = userService.getUserById((Integer) session.getAttribute("userId"));
122     model.addAttribute("catalog", catalog);
123     model.addAttribute("user", user.get());
124     return "createCatalog";
125 }
126
127 @PostMapping("/{createCatalog}")
128 public String addCatalog(@Valid @ModelAttribute("catalog") Catalog catalog, BindingResult bindingResult, Model model, HttpSession session) {
129     if (bindingResult.hasErrors()) {
130         model.addAttribute("errors", bindingResult);
131         return "createCatalog";
132     }
133     catalogService.saveCatalog(catalog, (Integer) session.getAttribute("userId"));
134     return "redirect:/catalogs";
135 }
```

Edycja:

```
149 @GetMapping("/{editCatalog/{catalogId}}")
150 public String redirectEdit(Model model, @PathVariable("catalogId") Integer catalogId, HttpSession session){
151     Integer userId = (Integer) session.getAttribute("userId");
152     Optional<Catalog> catalog = catalogService.getCatalogById(catalogId);
153     if (catalog.isPresent() && catalog.get().getUser().getId().equals(userId) && !(catalog.get().getName().equals("default")) &&
154         !(catalog.get().getName().equals("shared")))
155     {
156         model.addAttribute("catalog", catalog.get());
157         return "editCatalog";
158     }
159     return "redirect:/catalogs";
160 }
161
162 @PostMapping("/{editCatalog/{catalogId}}")
163 public String editCatalog(@Valid @ModelAttribute("catalog") Catalog catalog, BindingResult bindingResult,
164     @PathVariable("catalogId") Integer catalogId, Model model){
165
166     if (bindingResult.hasErrors()) {
167         model.addAttribute("errors", bindingResult);
168         catalog.setId(catalogId);
169         model.addAttribute("catalog", catalog);
170     }
171     return "editCatalog";
172 }
173 catalogService.updateCatalog(catalog, catalogId);
174 return "redirect:/catalogs";
175 }
```

Usuwanie:

```
137 @GetMapping("/{catalogId}")
138 public String delete(Model model, @PathVariable("catalogId") Integer catalogId, HttpSession session) {
139     Integer userId = (Integer)session.getAttribute("userId");
140     Optional<Catalog> catalog = catalogService.getCatalogById(catalogId);
141     if(catalog.isPresent() && catalog.get().getUser().getId().equals(userId) && !(catalog.get().getName().equals("default"))
142         && !(catalog.get().getName().equals("shared")))
143     {
144         catalogService.deleteCatalogById(catalogId);
145     }
146     return "redirect:/catalogs";
147 }
148
```

Metody wyłapujące żądania GET przekierowują odpowiednio na widoki **createCatalog.html** i **editCatalog.html**, które po wypełnieniu formularzy przekierowują na odpowiednie linki z metodami POST.

Wyświetlenie udostępnionych przez innych informacji - 2p.

Przy rejestracji użytkownika tworzone są dwa unikatowe (nieusuwalne i nieedytowalne) katalogi: **default** i **shared**. Katalog default jest katalogiem domyślnym, a w katalogu **shared** pokazywane są notatki udostępnione przez innych. Udostępnione notatki przechowywane są w bazie danych w osobnej tabeli **shared_notes**, która zawiera id udostępnianej notatki oraz id odbiorcy. Notatki w katalogu **shared** oprócz standardowych atrybutów mają też nazwę osoby udostępniającej.

Udostępnienie: ze wskazaniem na konkretnego użytkownika lub w linku - 1p.

Zaimplementowaliśmy możliwość udostępnienia notatki innemu użytkownikowi. Po kliknięciu przycisku udostępniania przy notatce, zostajemy przekierowani na widok z formularzem **shareNote.html**, na którym możemy podać użytkownika, któremu chcemy udostępnić notatkę. Pojawi się ona na jego koncie w katalogu **shared**.

Fragment kontrolera **NotesController.java** odpowiadające za udostępnianie:

```
374 @GetMapping("/{shareNote/{noteId}}")
375 public String redirectShare(Model model, @PathVariable("noteId") Integer noteId, @PathVariable("catalogId") Integer catalogId, HttpSession session) {
376     Optional<Catalog> catalog = catalogService.getCatalogById(catalogId);
377     Integer userId = (Integer) session.getAttribute("userId");
378     if (catalog.isPresent() && catalog.get().getUser().getId().equals(userId) && !(catalog.get().getName().equals("shared"))) {
379         List<User> userList = userService getUsers();
380         List<User> filteredUserList = new ArrayList<>();
381         Optional<Note> note = noteService.getNoteById(noteId);
382         for (User user : userList) {
383             if (!user.getId().equals(session.getAttribute("userId"))) {
384                 filteredUserList.add(user);
385             }
386         }
387         model.addAttribute("users", filteredUserList);
388         model.addAttribute("note", note.get());
389         model.addAttribute("catalog", catalog.get());
390         return "shareNote";
391     }
392     return "redirect:/catalogs";
393 }
394
395 @PostMapping("/{shareNote/{noteId}}")
396 public String shareNote(@RequestParam(value = "username") String username, @PathVariable("noteId") Integer noteId,
397     @PathVariable("catalogId") Integer catalogId, HttpSession session) {
398     Optional<User> user = userService.getUserByUsername(username);
399     if (user.isPresent() && !session.getAttribute("userId").equals(user.get().getId())) {
400         SharedNote sharedNote = new SharedNote();
401         sharedNote.setNote(noteService.getNoteById(noteId).get());
402         sharedNote.setUser(user.get());
403         noteService.saveSharedNote(sharedNote);
404     }
405     return "redirect:/catalogs/" + catalogId;
406 }
```

Wyświetlanie “swoich” informacji: sortowanie w obu kierunkach (data, kategoria, alfabetycznie, popularność) - 2p.

Zaimplementowaliśmy sortowanie katalogów po nazwie oraz popularności (liczbie notatek wewnątrz katalogu) oraz sortowanie notatek po nazwie i dacie. W widoku **catalogs.html** oraz **notes.html** znajdują się odpowiednie linki, dzięki którym możemy uzyskać posortowane informacje.

Fragment widoku katalogów **catalogs.html**:

```
17 <div class="side-title">SORT OPTIONS</div>
18 <div class="side-menu">
19     <a th:href="@{/catalogs/ASC}" th:value="1" ><i class="sort fa-solid fa-arrow-down-a-z"></i></a>
20     <a th:href="@{/catalogs/DESC}" th:value="2" ><i class="sort fa-solid fa-arrow-up-a-z"></i></a>
21     <a th:href="@{/catalogs/notesASC}" th:value="3" ><i class="sort fa-solid fa-arrow-down-1-9"></i></a>
22     <a th:href="@{/catalogs/notesDESC}" th:value="4" ><i class="sort fa-solid fa-arrow-up-1-9"></i></a>
23     <a th:href="@{/catalogs/deleteFilters}" th:value="5" ><i class="fa-solid fa-filter-circle-xmark"></i></a>
24 </div>
```

Fragment widoku notatek **notes.html**:

```
18 <div class="side-menu">
19     <a th:href="@{/catalogs/ + ${catalog.getId()}/ASC}" th:value="1" ><i class="sort fa-solid fa-arrow-down-a-z"></i></a>
20     <a th:href="@{/catalogs/ + ${catalog.getId()}/DESC}" th:value="2" ><i class="sort fa-solid fa-arrow-up-a-z"></i></a>
21     <a th:href="@{/catalogs/ + ${catalog.getId()}/dataASC}" th:value="3" ><i class="sort fa-regular fa-calendar-plus"></i></a>
22     <a th:href="@{/catalogs/ + ${catalog.getId()}/dataDESC}" th:value="4" ><i class="sort fa-regular fa-calendar-minus"></i></a>
23     <a th:href="@{/catalogs/ + ${catalog.getId()}/deleteFilters}" th:value="5" ><i class="fa-solid fa-filter-circle-xmark"></i></a>
```

Fragmenty kontrolera **CatalogsController.java**:

```
54 @GetMapping("/{ASC}")
55 public String sortCatalogsASC(Model model, HttpSession session, HttpServletResponse response, HttpServletRequest request) {
56     Optional<Cookie> cookie = Arrays.stream(request.getCookies()).filter(
57         c -> c.getName().equals("catalogCookie" + session.getAttribute("userId"))).findAny();
58     cookie.get().setValue("ASC");
59     response.addCookie(cookie.get());
60     List<Catalog> catalogList = catalogService.getCatalogsByUserId((Integer) session.getAttribute("userId"));
61     catalogList.sort(Comparator.comparing(Catalog::getName));
62     Optional<User> user = userService.getUserById((Integer) session.getAttribute("userId"));
63     model.addAttribute("catalogs", catalogList);
64     model.addAttribute("user", user.get());
65     return "catalogs";
66 }
67
68 @GetMapping("/{DESC}")
69 public String sortCatalogsDESC(Model model, HttpSession session, HttpServletResponse response, HttpServletRequest request) {
70     Optional<Cookie> cookie = Arrays.stream(request.getCookies()).filter(
71         c -> c.getName().equals("catalogCookie" + session.getAttribute("userId"))).findAny();
72     cookie.get().setValue("DESC");
73     response.addCookie(cookie.get());
74     List<Catalog> catalogList = catalogService.getCatalogsByUserId((Integer) session.getAttribute("userId"));
75     catalogList.sort(Comparator.comparing(Catalog::getName).reversed());
76     Optional<User> user = userService.getUserById((Integer) session.getAttribute("userId"));
77     model.addAttribute("catalogs", catalogList);
78     model.addAttribute("user", user.get());
79     return "catalogs";
80 }
81
82 @GetMapping("/{notesASC}")
83 public String sortCatalogsByNotesASC(Model model, HttpSession session, HttpServletRequest request, HttpServletResponse response) {
84     Optional<Cookie> cookie = Arrays.stream(request.getCookies()).filter(
85         c -> c.getName().equals("catalogCookie" + session.getAttribute("userId"))).findAny();
86     cookie.get().setValue("notesASC");
87     response.addCookie(cookie.get());
88     List<Catalog> catalogList = catalogService.getCatalogsByUserId((Integer) session.getAttribute("userId"));
89     catalogList.sort(Comparator.comparing(Catalog::getNoteCount));
90     Optional<User> user = userService.getUserById((Integer) session.getAttribute("userId"));
91     model.addAttribute("catalogs", catalogList);
92     model.addAttribute("user", user.get());
93     return "catalogs";
94 }
95
96 @GetMapping("/{notesDESC}")
97 public String sortCatalogsByNotesDESC(Model model, HttpSession session, HttpServletRequest request, HttpServletResponse response) {
98     Optional<Cookie> cookie = Arrays.stream(request.getCookies()).filter(
99         c -> c.getName().equals("catalogCookie" + session.getAttribute("userId"))).findAny();
100     cookie.get().setValue("notesDESC");
101     response.addCookie(cookie.get());
102     List<Catalog> catalogList = catalogService.getCatalogsByUserId((Integer) session.getAttribute("userId"));
103     catalogList.sort(Comparator.comparing(Catalog::getNoteCount).reversed());
104     Optional<User> user = userService.getUserById((Integer) session.getAttribute("userId"));
105     model.addAttribute("catalogs", catalogList);
106     model.addAttribute("user", user.get());
107     return "catalogs";
108 }
109
110 @GetMapping("/deleteFilters")
111 public String deleteFilters(Model model, HttpSession session, HttpServletRequest request, HttpServletResponse response) {
112     Optional<Cookie> cookie = Arrays.stream(request.getCookies()).filter(
113         c -> c.getName().equals("catalogCookie" + session.getAttribute("userId"))).findAny();
114     cookie.get().setValue("default");
115     response.addCookie(cookie.get());
116     return "redirect:/catalogs";
117 }
```

Fragmenty kontrolera **NotesController.java**:

```
89     @GetMapping("/{ASC}")
90     public String sortNotesByTitleASC(Model model, @PathVariable("catalogId") Integer catalogId,
91                                     HttpSession session, HttpServletResponse response, HttpServletRequest request) {
92
93         Optional<Catalog> catalog = catalogService.getCatalogById(catalogId);
94         Integer userId = (Integer) session.getAttribute("userId");
95
96         if (catalog.isPresent() && catalog.get().getUser().getId().equals(userId)) {
97             Optional<Cookie> cookie = Arrays.stream(request.getCookies()).filter(
98                 c -> c.getName().equals("noteCookie" + session.getAttribute("userId")))
99                 .findAny();
100             cookie.get().setValue("ASC");
101             response.addCookie(cookie.get());
102
103             List<Note> noteList;
104             if (catalog.get().getName().equals("shared")) {
105                 noteList = noteService.getNotesFromSharedByCatalogId(userId);
106             } else {
107                 noteList = noteService.getNotesByCatalogId(catalogId);
108             }
109             LocalDate oldDate = null;
110             LocalDate newDate = null;
111
112             for (Note n : noteList) {
113                 LocalDate currentDate = n.getDate();
114
115                 if (oldDate == null || currentDate.isBefore(oldDate)) {
116                     oldDate = currentDate;
117                 }
118
119                 if (newDate == null || currentDate.isAfter(newDate)) {
120                     newDate = currentDate;
121                 }
122             }
123
124             if (oldDate == null && newDate == null) {
125                 oldDate = LocalDate.now();
126                 newDate = LocalDate.now();
127             }
128
129             Optional<User> user = userService.getUserById((Integer) session.getAttribute("userId"));
130             noteList.sort(Comparator.comparing(Note::getTitle));
131             model.addAttribute("oldDate", oldDate);
132             model.addAttribute("newDate", newDate);
133             model.addAttribute("sharedNotes", noteService.getMySharedNotes(userId));
134             model.addAttribute("catalog", catalog.get());
135             model.addAttribute("notes", noteList);
136             model.addAttribute("user", user.get());
137             return "notes";
138         }
139         return "redirect:/catalogs";
140     }
141
142     @GetMapping("/{DESC}")
143     public String sortNotesByTitleDESC(Model model, @PathVariable("catalogId") Integer catalogId,
144                                       HttpSession session, HttpServletResponse response, HttpServletRequest request) {
145         Optional<Catalog> catalog = catalogService.getCatalogById(catalogId);
146         Integer userId = (Integer) session.getAttribute("userId");
147
148         if (catalog.isPresent() && catalog.get().getUser().getId().equals(userId)) {
149             Optional<Cookie> cookie = Arrays.stream(request.getCookies()).filter(
150                 c -> c.getName().equals("noteCookie" + session.getAttribute("userId")))
151                 .findAny();
152             cookie.get().setValue("DESC");
153             response.addCookie(cookie.get());
154
155             List<Note> noteList;
156             if (catalog.get().getName().equals("shared")) {
157                 noteList = noteService.getNotesFromSharedByCatalogId(userId);
158             } else {
159                 noteList = noteService.getNotesByCatalogId(catalogId);
160             }
161
162             LocalDate oldDate = null;
163             LocalDate newDate = null;
```

```

162
163     for (Note n : noteList) {
164         LocalDate currentDate = n.getDate();
165
166         if (oldDate == null || currentDate.isBefore(oldDate)) {
167             oldDate = currentDate;
168         }
169
170         if (newDate == null || currentDate.isAfter(newDate)) {
171             newDate = currentDate;
172         }
173     }
174
175     if(oldDate == null && newDate == null){
176         oldDate = LocalDate.now();
177         newDate = LocalDate.now();
178     }
179
180     Optional<User> user = userService.getUserById((Integer) session.getAttribute("userId"));
181     noteList.sort(Comparator.comparing(Note::getTitle).reversed());
182     model.addAttribute("oldDate", oldDate);
183     model.addAttribute("newDate", newDate);
184     model.addAttribute("sharedNotes", noteService.getMySharedNotes(userId));
185     model.addAttribute("catalog", catalog.get());
186     model.addAttribute("notes", noteList);
187     model.addAttribute("user", user.get());
188     return "notes";
189 }
190 return "redirect:/catalogs";
191 }
192
193 @GetMapping("/dataASC")
194 public String sortNotesByDateASC(Model model, @PathVariable("catalogId") Integer catalogId,
195     HttpSession session, HttpServletResponse response, HttpServletRequest request) {
196     Optional<Catalog> catalog = catalogService.getCatalogById(catalogId);
197     Integer userId = (Integer) session.getAttribute("userId");
198

```

```

198
199     if (catalog.isPresent() && catalog.get().getUser().getId().equals(userId)) {
200         Optional<Cookie> cookie = Arrays.stream(request.getCookies()).filter(
201             c -> c.getName().equals("noteCookie" + session.getAttribute("userId")))
202             .findAny();
203         cookie.get().setValue("dataASC");
204         response.addCookie(cookie.get());
205         List<Note> noteList;
206         if (catalog.get().getName().equals("shared")) {
207             noteList = noteService.getNotesFromSharedByCatalogId(userId);
208         } else {
209             noteList = noteService.getNotesByCatalogId(catalogId);
210         }
211         LocalDate oldDate = null;
212         LocalDate newDate = null;
213         for (Note n : noteList) {
214             LocalDate currentDate = n.getDate();
215             if (oldDate == null || currentDate.isBefore(oldDate)) {
216                 oldDate = currentDate;
217             }
218
219             if (newDate == null || currentDate.isAfter(newDate)) {
220                 newDate = currentDate;
221             }
222         }
223         if(oldDate == null && newDate == null){
224             oldDate = LocalDate.now();
225             newDate = LocalDate.now();
226         }
227         Optional<User> user = userService.getUserById((Integer) session.getAttribute("userId"));
228         noteList.sort(Comparator.comparing(Note::getDate));
229         model.addAttribute("oldDate", oldDate);
230         model.addAttribute("newDate", newDate);
231         model.addAttribute("sharedNotes", noteService.getMySharedNotes(userId));
232         model.addAttribute("catalog", catalog.get());
233         model.addAttribute("notes", noteList);
234         model.addAttribute("user", user.get());
235         return "notes";
236     }
237     return "redirect:/catalogs";

```

Zapamiętanie kierunków i kryteriów sortowania - 1p.

Do zapamiętywania kierunków sortowania katalogów oraz notatek używamy ciasteczek. Tworzone są one przy rejestracji, a ustawiane podczas wyboru kierunku sortowania (co widać na zdjęciach w punkcie powyżej). Kontrolery **NotesController.java** oraz **CatalogsController.java** odczytują te ciasteczka i przekierowują na odpowiednie linki.

Fragment kontrolera **CatalogsController.java**:

```
31 @GetMapping("")
32 public String getCatalogsByUserId(Model model, HttpSession session, HttpServletRequest request) {
33     Optional<String> sortValue = Arrays.stream(request.getCookies()).filter(
34         c -> c.getName().equals("catalogCookie" + session.getAttribute("userId"))).map(Cookie::getValue).findAny();
35
36     if(sortValue.isEmpty())
37         return "redirect:/login";
38
39     if (sortValue.get().equals("ASC"))
40         return "redirect:/catalogs/ASC";
41     else if (sortValue.get().equals("DESC"))
42         return "redirect:/catalogs/DESC";
43     else if (sortValue.get().equals("notesASC"))
44         return "redirect:/catalogs/notesASC";
45     else if (sortValue.get().equals("notesDESC"))
46         return "redirect:/catalogs/notesDESC";
47 }
```

Fragment kontrolera **NotesController.java**:

```
32 @GetMapping("")
33 public String getNotesByCatalogId(Model model, @PathVariable("catalogId") Integer catalogId, HttpSession session, HttpServletRequest request) {
34     Optional<Catalog> catalog = catalogService.getCatalogById(catalogId);
35     Integer userId = (Integer) session.getAttribute("userId");
36
37     Optional<String> sortValue = Arrays.stream(request.getCookies()).filter(
38         c -> c.getName().equals("noteCookie" + session.getAttribute("userId"))).map(Cookie::getValue).findAny();
39
40     if (catalog.isPresent() && catalog.get().getUser().getId().equals(userId)) {
41         if (sortValue.get().equals("ASC"))
42             return "redirect:/catalogs/{catalogId}/ASC";
43         else if (sortValue.get().equals("DESC"))
44             return "redirect:/catalogs/{catalogId}/DESC";
45         else if (sortValue.get().equals("dataASC"))
46             return "redirect:/catalogs/{catalogId}/dataASC";
47         else if (sortValue.get().equals("dataDESC"))
48             return "redirect:/catalogs/{catalogId}/dataDESC";
49     }
```

Filtrowanie - 2p.

Zaimplementowaliśmy filtrowanie notatek po dacie. Użytkownik może wybrać datę początkową i końcową, następnie następuje przekierowanie na link, gdzie lista notatek zostaje przefiltrowana oraz pokazana. Został również utworzony pasek wyszukiwania pozwalający znajdować notatki lub katalogi po nazwie.

Fragment widoku **notes.html** z formularzem do wyboru daty:

```
28 <form class="dates" th:action="@{/catalogs/{catalogId}/filterByDates}" th:method="post">
29     <label class="label-date">
30         <span class="date-note">From: </span>
31         <input type="date" name="startDate" th:value="@{oldDate}" />
32     </label>
33     <label class="label-date">
34         <span class="date-note">To: </span>
35         <input type="date" name="endDate" th:value="@{newDate}" />
36     </label>
37     <button class="filter" type="submit"><i class="fa-solid fa-filter"></i></button>
38 </form>
39 <a th:if="@{catalog.getName() != 'shared'}" th:href="@{/catalogs/{catalogId}/createNote}"
40 ><i class="fa-regular fa-square-plus"></i>
41 </a>
```

Fragment kontrolera **NotesController.java**:

```
467 @PostMapping("/filterByDates")
468 public String filter(@RequestParam(name="startDate") LocalDate startDate,
469                     @RequestParam(name="endDate") LocalDate endDate, Model model, @PathVariable("catalogId") Integer catalogId, HttpSession session){
470
471     Optional<Catalog> catalog = catalogService.getCatalogById(catalogId);
472     Integer userId = (Integer) session.getAttribute("userId");
473     if (catalog.isPresent() && catalog.get().getUser().getId().equals(userId)) {
474         List<Note> noteList;
475         if (catalog.get().getName().equals("shared")) {
476             noteList = noteService.getNotesFromSharedByCatalogId(userId);
477         } else {
478             noteList = noteService.getNotesByCatalogId(catalogId);
479         }
480         Optional<User> user = userService.getUserById((Integer) session.getAttribute("userId"));
481         model.addAttribute("oldDate", startDate);
482         model.addAttribute("newDate", endDate);
483         model.addAttribute("user", user.get());
484         model.addAttribute("sharedNotes", noteService.getMySharedNotes(userId));
485         model.addAttribute("catalog", catalog.get());
486         model.addAttribute("notes", noteList.stream().filter(n
487             -> (n.getDate().isAfter(startDate) && n.getDate().isBefore(endDate))
488             || n.getDate().isEqual(startDate) || n.getDate().isEqual(endDate)).toList());
489         return "notes";
490     }
491     return "redirect:/catalogs";
492 }
```

Logowanie - 1p.

Logowanie jest oparte na formularzu zawartym w **login.html**. Autentykacją użytkownika zajmuje się Spring Security, który został przez nas odpowiednio skonfigurowany.

Fragment widoku **login.html**:

```
17 <div class="login-box">
18     <form th:action="@{/login}" method="post" role="form">
19         <h2 class="note">Note<span class="ally">ally</span></h2>
20         <div class="input-box">
21             <span><i class="fa-sharp fa-solid fa-user"></i></span>
22             <input type="text" id="login" name="login" class="form-control" />
23             <label for="login">Login<div class="error-label-name" th:if="${session.info!=null}
24                 && !session.info.equals('')>" th:text="${session.info}></div></label>
25         </div>
```

Fragment **Log_Reg_Controller.java**:

```
26
27 @GetMapping("/login")
28 public String redirectLogin(HttpSession session){
29     if(session.getAttribute("userId") != null)
30     {
31         return "redirect:/catalogs";
32     }
33     return "login";
34 }
35
```

Fragment pliku konfiguracyjnego modułu Spring Security **WebSecurity.java**:

```
43     ).formLogin(  
44         form -> form  
45             .loginPage("/login")  
46             .usernameParameter("login")  
47             .passwordParameter("password")  
48             .loginProcessingUrl("/login")  
49             .permitAll()  
50             .defaultSuccessUrl("/catalogs")  
51             .failureHandler(new AuthenticationFailureHandler() {  
52                 @Override  
53                 public void onAuthenticationFailure(HttpServletRequest request, HttpServletResponse response, AuthenticationException exception)  
54                     throws IOException, ServletException {  
55                     HttpSession session = request.getSession();  
56                     session.setAttribute("info", "Incorrect Login or Password");  
57                     response.sendRedirect("/login");  
58                 }  
59             })  
60     ).sessionManagement(  
61         session -> session  
62             .sessionCreationPolicy(SessionCreationPolicy.ALWAYS) SessionManagementConfigurer<HttpSecurity>  
63             .invalidSessionUrl("/login")  
64             .maximumSessions(1) ConcurrencyControlConfigurer  
65             .maxSessionsPreventsLogin(false)  
66     )
```

Dodatkowo został przez nas zaimplementowany dodatkowy “listener” **AuthenticationApplicationListener.java**, który przy udanej autentykacji dodaje do sesji atrybut **userId**, który jest potem wykorzystywany np. do pobrania odpowiednich katalogów po id użytkownika.

AuthenticationApplicationListener.java:

```
1  package pl.noteally.config;  
2  
3  
4  import jakarta.servlet.http.HttpSession;  
5  import lombok.AllArgsConstructor;  
6  import org.springframework.context.event.EventListener;  
7  import org.springframework.security.authentication.event.InteractiveAuthenticationSuccessEvent;  
8  import org.springframework.stereotype.Component;  
9  import pl.noteally.data.User;  
10  
11  @Component  
12  @AllArgsConstructor  
13  public class AuthenticationApplicationListener {  
14  
15      private final HttpSession session;  
16      @EventListener  
17      public void handleInteractiveAuthenticationSuccess(InteractiveAuthenticationSuccessEvent event) {  
18          User user = (User) event.getAuthentication().getPrincipal();  
19          session.setAttribute("userId", user.getId());  
20      }  
21  }
```

Zapis do bazy danych dopiero przy wylogowaniu/wygaśnięciu sesji - 2p.

Ten podpunkt nie został zaimplementowany, ponieważ uznaliśmy, że zaburzyłoby to działanie naszej aplikacji.

IMPLEMENTACJA WYMAGAŃ UŻYTKOWNIKA LIMITED

W klasie **WebSecurity.java** są zdefiniowane odpowiednie uprawnienia dla poszczególnych użytkowników. Użytkownik o roli "LIMITED USER" ma ograniczone uprawnienia dostępu, które nie obejmują tworzenia ani udostępniania notatek.

Fragment pliku konfiguracyjnego **WebSecurity.java**:

```
43 @Bean
44 public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
45     http
46         .authorizeHttpRequests(authz -> authz
47             .requestMatchers("/css/**", "/images/**", "/js/**").permitAll()
48             .requestMatchers("/register", "/", "/login").permitAll()
49             .requestMatchers("/admin/**").hasAuthority("ADMIN")
50             .requestMatchers("/catalogs/createCatalog", "/catalogs/deleteCatalog", "/catalogs/editCatalog").
51             hasAnyAuthority("ADMIN", "USER")
52             .requestMatchers("/catalogs/{catalogId}/").hasAnyAuthority("ADMIN", "USER")
53             .anyRequest().authenticated()
54         )
55 }
```

IMPLEMENTACJA WYMAGAŃ Z PUNKTU 3 (NIEZALOGOWANY)

Rejestracja - 1p.

Rejestracja odbywa się za pomocą formularza w widoku **register.html**, który tworzy obiekt **User** i przesyła go na odpowiedni link z metodą POST. W kontrolerze **Log_Reg_Controller.java** odbywa się również sprawdzanie błędów walidacji oraz zapytanie do utworzonego przez nas serwera REST o nazwie **isNamePolish**, który sprawdza, czy podane imię jest polskie.

Fragment formularza w widoku **register.html**:

```
44 <form th:action="@{/register}" th:method="post" th:object="${user}">
45     <h2 class="note">Registration</h2>
46     <div class="input-box">
47         <span><i class="fa-sharp fa-solid fa-user"></i></span>
48         <input type="text" id="username" name="username" th:field="*{login}" th:value="${username}"/>
49         <label for="username">Username</label>
50         <div class="error-label-name">
51             <span th:if="${#fields.hasErrors('login')}" th:errors="*{login}"></span>
52             <span th:if="${usernameTaken}" th:text="${usernameTaken}"></span>
53         </div>
54     </div>
55     <div class="input-box">
56         <span><i class="fa-sharp fa-solid fa-signature"></i></span>
57         <input type="text" id="name" name="name" th:field="*{name}" th:value="${name}"/>
58         <label for="name">Name</label>
59         <div class="error-label-name">
60             <span th:if="${#fields.hasErrors('name')}" th:errors="*{name}"></span>
61             <span th:if="${polishNameError}" th:text="${polishNameError}"></span>
62         </div>
63     </div>
64     <div class="input-box">
65         <span><i class="fa-sharp fa-solid fa-signature"></i></span>
66         <input type="text" id="surname" name="surname" th:field="*{surname}" th:value="${surname}"/>
67         <label for="surname">Surname</label>
68         <div class="error-label-name">
69             <span th:if="${#fields.hasErrors('surname')}" th:errors="*{surname}"></span>
70         </div>
71     </div>
72     <div class="input-box">
73         <span><i class="fa-sharp fa-solid fa-calendar"></i></span>
74         <input type="number" id="age" name="age" th:field="*{age}" th:value="${age}"/>
75         <label for="age">Age</label>
76         <div class="error-label-name">
77             <span th:if="${#fields.hasErrors('age')}" th:errors="*{age}"></span>
78         </div>
79     </div>
80     <div class="input-box">
81         <span><i class="fa-sharp fa-solid fa-lock"></i></span>
82         <input type="password" id="password" name="password" th:field="*{password}" th:value="${password}"/>
83         <label for="password">Password</label>
84         <div class="error-label-name">
85             <span th:if="${#fields.hasErrors('password')}" th:errors="*{password}"></span>
86         </div>
87     </div>
88 </form>
```

Fragmenty kontrolera **Log_Reg_Controller.java**:

```
36     @GetMapping("/register")
37     public String redirectRegister(Model model, HttpSession session){
38         if(session.getAttribute("userId") != null)
39         {
40             return "redirect:/catalogs";
41         }
42
43         model.addAttribute(attributeName: "user", new User());
44         return "register";
45     }
```

```
47     @PostMapping("/register")
48     public String register(@Valid @ModelAttribute("user") User user, BindingResult bindingResult, @RequestParam("confirmPassword") String confirmPassword,
49                           HttpServletResponse response, Model model) {
50         boolean errors = false;
51         model.addAttribute(attributeName: "user", user);
52         model.addAttribute(attributeName: "username", user.getUsername());
53         model.addAttribute(attributeName: "name", user.getName());
54         model.addAttribute(attributeName: "surname", user.getSurname());
55         model.addAttribute(attributeName: "age", user.getAge());
56         model.addAttribute(attributeName: "password", user.getPassword());
57
58         if(userService.userExists(user)) {
59             model.addAttribute(attributeName: "usernameTaken", attributeValue: "This username is already taken");
60             errors = true;
61         }
62         if(!user.getPassword().equals(confirmPassword)) {
63             model.addAttribute(attributeName: "confirmError", attributeValue: "Passwords are different");
64             errors = true;
65         }
66         if(!restNameService.isNamePolish(user.getName())) {
67             model.addAttribute(attributeName: "polishNameError", attributeValue: "Name must be Polish");
68             errors = true;
69         }
70         if (bindingResult.hasErrors()) {
71             model.addAttribute(attributeName: "errors", bindingResult);
72             errors = true;
73         }
74         if(errors) return "register";
75
76         userService.signUpUser(user);
77
78         Cookie catalogCookie = new Cookie(name: "catalogCookie" + user.getId(), value: "default");
79         catalogCookie.setMaxAge(60 * 60 * 24 * 365 * 10);
80         Cookie noteCookie = new Cookie(name: "noteCookie" + user.getId(), value: "default");
81         noteCookie.setMaxAge(60 * 60 * 24 * 365 * 10);
82         response.addCookie(catalogCookie);
83         response.addCookie(noteCookie);
84
85         return "login";
86     }
```

Fragment serwisu **UserService.java**:

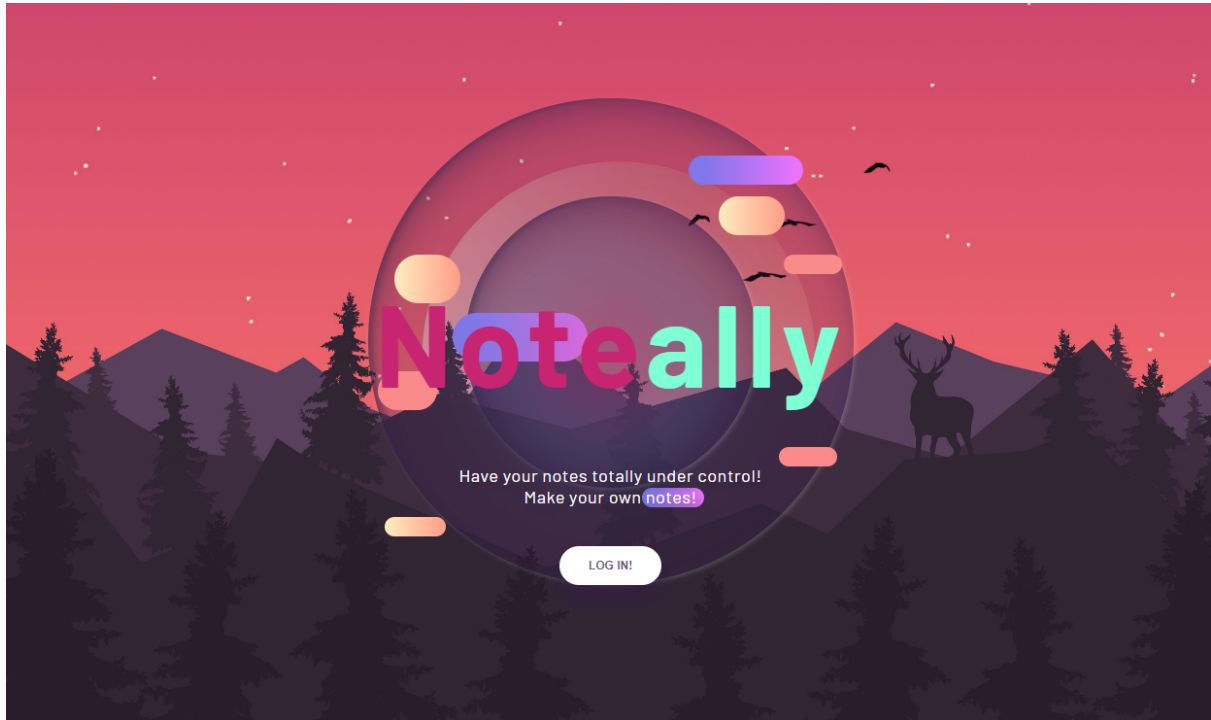
```
38 @ public String signUpUser(User user){
39
40     String encodedPassword = bCryptPasswordEncoder.encode(user.getPassword());
41
42     user.setPassword(encodedPassword);
43
44     userRepository.save(user);
45
46     //default catalogs
47     Catalog defaultCatalog = new Catalog();
48     defaultCatalog.setName("default");
49     defaultCatalog.setUser(user);
50     catalogRepository.save(defaultCatalog);
51
52     Catalog sharedCatalog = new Catalog();
53     sharedCatalog.setName("shared");
54     sharedCatalog.setUser(user);
55     catalogRepository.save(sharedCatalog);
56
57     return "catalogs";
58 }
```

Walidacja formularza - 1p.

Walidacja formularza rejestracji została opisana wcześniej (punkt "Walidacja Formularza" przy opisywaniu użytkownika z pełnymi prawami)

Strona powitalna - 1p.

Widok strony powitalnej jest dostępny dla każdego użytkownika:



Wyświetlenie informacji z udostępnionego linku - 1p.

Nie zaimplementowaliśmy udostępniania notatek przez link, ponieważ kłóciło się to z naszą wizją projektu. Notatkę możemy udostępnić tylko innemu, zalogowanemu użytkownikowi.

IMPLEMENTACJA WYMAGAŃ Z PUNKTU 4 (ADMIN)

Wyśw. listy użytkowników - 1p.

W panelu Admina, do którego dostęp ma tylko użytkownik o roli "ADMIN", wyświetla się lista użytkowników (można ją sortować alfabetycznie). Admin może edytować i usuwać użytkowników.

Fragment widoku **admin.html**:

```
36      <ul th:each="user : ${userList}">
37          <div class="container-user">
38              <li>
39                  <i class="fa-solid fa-user"></i>
40                  <p>
41                      th:href="@{'/admin/' + ${user.getId()}}"
42                      th:text="${user.getLogin()}"
43                  >Username</p>
44              >
45          </li>
46          <a th:href="@{'/admin/' + 'deleteUser/' + ${user.getId()}}"
47              ><i class="fa-solid fa-xmark"></i>
48          </a>
49
50          <a th:href="@{'/admin/' + 'editUser/' + ${user.getId()}}"
51              ><i class="fa-regular fa-pen-to-square"></i>
52          </a>
53      </div>
54  </ul>
```

Fragment kontrolera **AdminController.java**:

```
24  @GetMapping("/")
25  public String getAllUser(Model model, HttpSession session) {
26      List<User> userList=userService getUsers();
27      List<User> filteredUserList = new ArrayList<>();
28      for (User user : userList) {
29          if (!user.getId().equals(session.getAttribute( s: "userId"))) {
30              filteredUserList.add(user);
31          }
32      }
33      model.addAttribute( attributeName: "userList", filteredUserList);
34      return "admin";
35  }
36  }
```

Zarządzanie rolami - 1p.

Admin może edytować wszystkie dane użytkowników (w tym rolę). Po wybraniu przycisku edycji przy użytkowniku, zostaje przekierowany na formularz edycji **editUser.html**.

Fragment widoku **editUser.html** odpowiedzialny za wybór nowej roli użytkownika:

```
81 <div class="input-box">
82   <select id="role" name="role" th:field="*{role}">
83     <option value="ADMIN">ADMIN</option>
84     <option value="USER">USER</option>
85     <option value="LIMITED">LIMITED</option>
86   </select>
87   <label for="role">Role</label>
88 </div>
```

Fragment kontrolera **AdminController.java**:

```
64 @GetMapping("/{userId}")
65 public String delete(Model model, @PathVariable("userId") Integer userId, HttpSession session) {
66     if((session.getAttribute("userId")).equals(userId))
67     {
68         return "redirect:/admin";
69     }
70     userService.deleteUserById(userId);
71     return "redirect:/admin";
72 }
73 @GetMapping("/{userId}/editUser/{userId}")
74 public String redirectEdit(Model model, @PathVariable("userId") Integer userId, HttpSession session){
75     if((session.getAttribute("userId")).equals(userId))
76     {
77         return "redirect:/admin";
78     }
79     Optional<User> user = userService.getUserById(userId);
80     model.addAttribute("user", user.get());
81     return "editUser";
82 }
83
84 @PostMapping("/{userId}/editUser/{userId}")
85 public String editUser(@Valid @ModelAttribute("user") User user, BindingResult bindingResult, @PathVariable("userId") Integer userId, Model model){
86
87     if (bindingResult.hasErrors()) {
88         user.setId(userId);
89         model.addAttribute("user", user);
90         model.addAttribute("errors", bindingResult);
91         return "editUser";
92     }
93     userService.updateUser(user, userId);
94     return "redirect:/admin";
95 }
96 }
97 }
```

Fragment serwisu **UserService.java**:

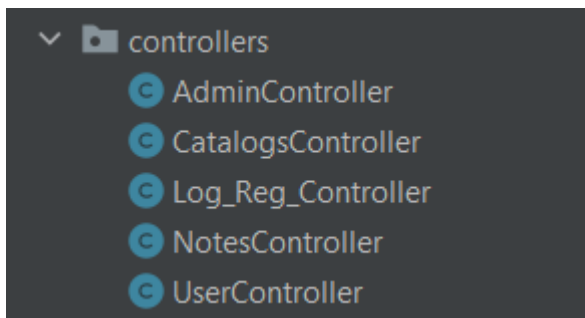
```
70 @
71 public void updateUser(User user, Integer userId)
72 {
73     Optional<User> existingUser = userRepository.findById(userId);
74     existingUser.get().setLogin(user.getLogin());
75     existingUser.get().setName(user.getName());
76     existingUser.get().setSurname(user.getSurname());
77     existingUser.get().setAge(user.getAge());
78     existingUser.get().setRole(user.getRole());
79     userRepository.save(existingUser.get());
80 }
```

IMPLEMENTACJA WYMAGAŃ Z PUNKTU 5 (ELEMENTY TECHNICZNE)

Kontrolery - 2p.

We wcześniejszych punktach dokumentacji opisano działanie poszczególnych kontrolerów, które obsługują określone żądania użytkowników.

Lista utworzonych przez nas kontrolerów:



5 różnych znaczników Thymeleafa - 3p.+2p.

Lista wszystkich znaczników Thymeleafa użytych w projekcie:

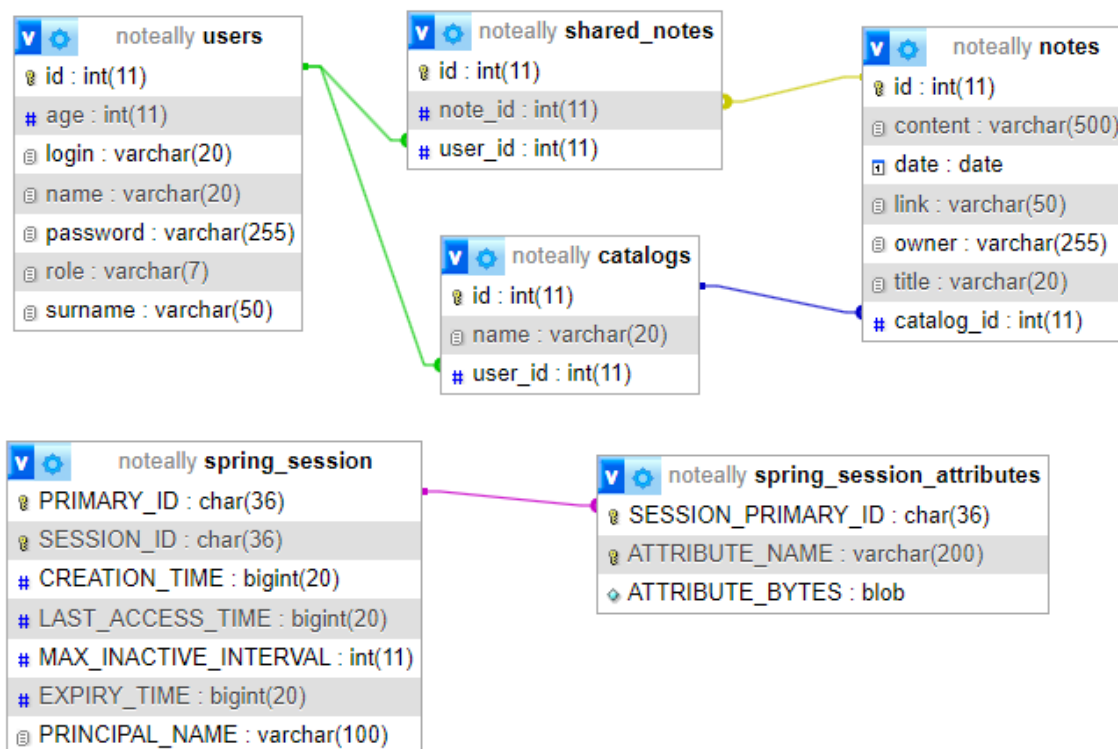
- th:href
- th:each
- th:if
- th:field
- th:src
- th:method
- th:object
- th:text
- th:value

Wykorzystanie thymeleafa w plikach .html umożliwiło nam dynamiczną interakcję frontendu z backendem.

Baza danych (co najmniej 2 tabele z relacją) - 5p.

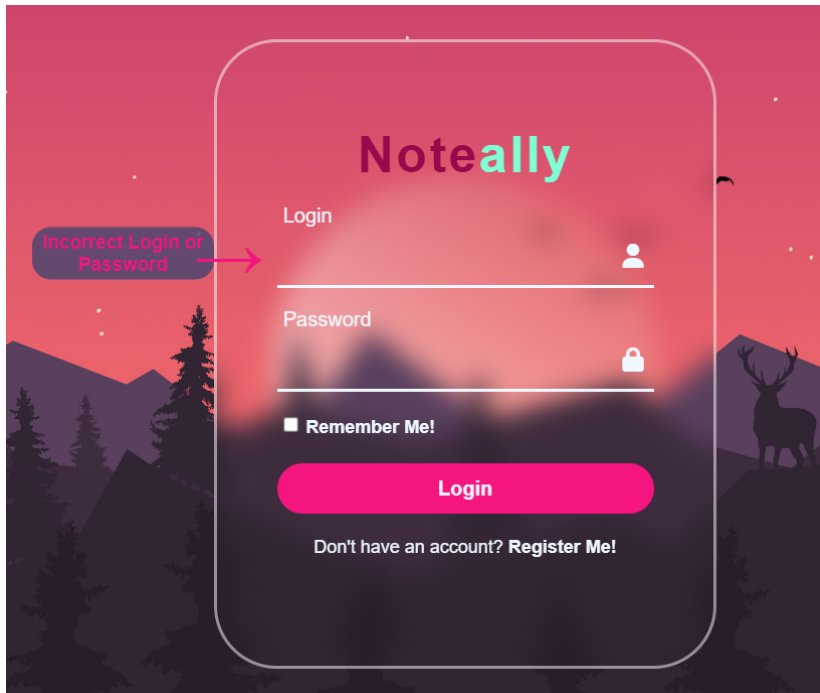
Do wykonania projektu wykorzystaliśmy bazę danych **MySQL**. Posiada ona 4 tabele utworzone na podstawie klas danych w projekcie (**users**, **catalogs**, **notes**, **shared_notes**) oraz 2 tabele tworzone przez Springa do zarządzania sesją użytkownika (**spring_session**, **spring_session_attributes**).

Schemat bazy danych z panelu **phpMyAdmin**:



Widoki: formularze z walidacją (3 różne elementy)

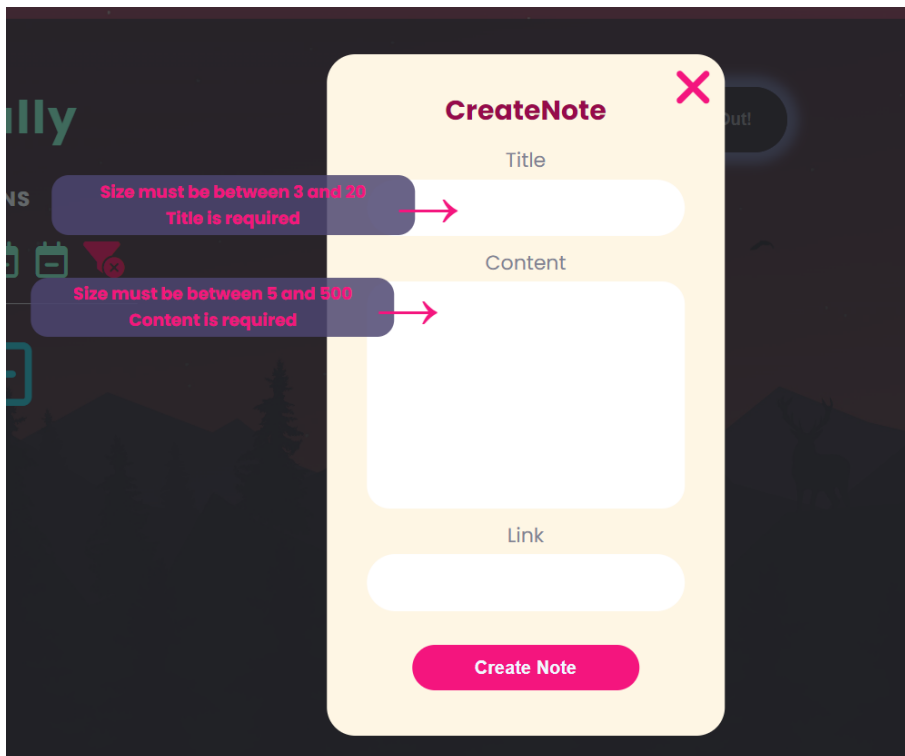
Przykład widoku z walidacją `login.html`:



The image shows a login form for 'Noteally' with a pink and purple color scheme. The form is centered on a background with a silhouette of a deer and mountains. The form contains the following elements:

- Header:** 'Noteally' logo in pink and green.
- Form Fields:**
 - Login:** A text input field with a pink user icon on the right. A pink error message 'Incorrect Login or Password' with an arrow points to this field.
 - Password:** A text input field with a pink lock icon on the right.
- Remember Me:** A checkbox labeled 'Remember Me!'.
- Login Button:** A pink button labeled 'Login'.
- Register Link:** A link labeled 'Don't have an account? Register Me!'.

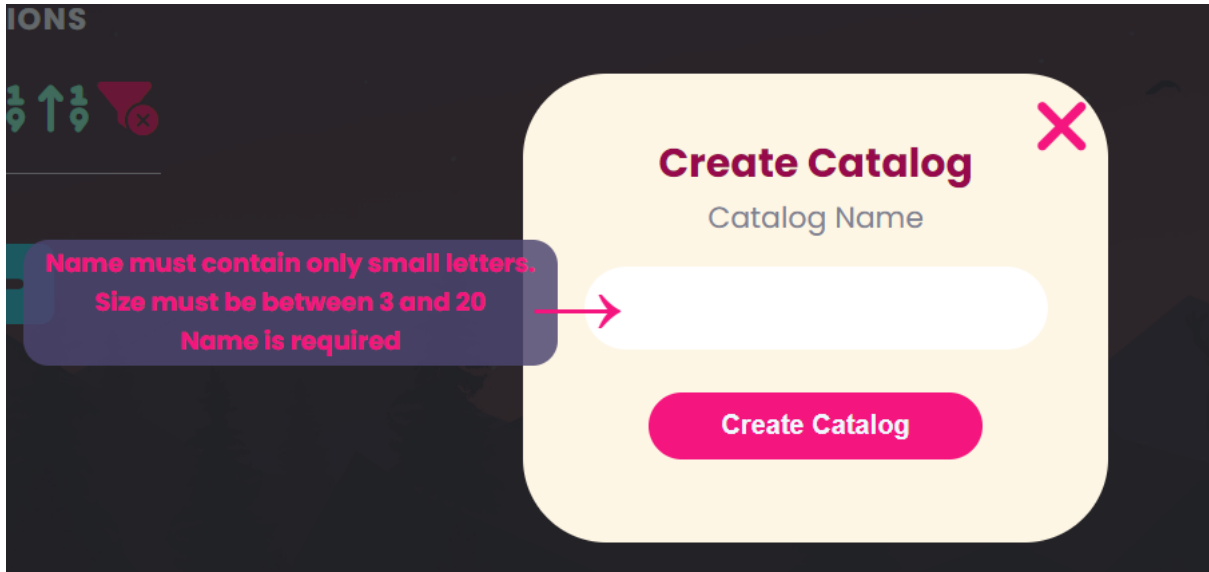
Przykład widoku z walidacją `createNote.html`:



The image shows a 'CreateNote' form for 'Noteally' with a pink and purple color scheme. The form is centered on a background with a silhouette of a deer and mountains. The form contains the following elements:

- Header:** 'CreateNote' title in pink, with a pink close button (X) on the right.
- Form Fields:**
 - Title:** A text input field. A pink error message 'Size must be between 3 and 20 Title is required' with an arrow points to this field.
 - Content:** A text area. A pink error message 'Size must be between 5 and 500 Content is required' with an arrow points to this field.
 - Link:** A text input field.
- Create Note Button:** A pink button labeled 'Create Note'.

Przykład widoku z walidacją **createCatalog.html**:



The screenshot shows a web form titled "Create Catalog" with a red 'X' icon in the top right corner. The form has a label "Catalog Name" above a text input field. Below the input field, there are three validation error messages in a red box: "Name must contain only small letters.", "Size must be between 3 and 20", and "Name is required". A red arrow points from the first error message to the input field. Below the input field is a red button labeled "Create Catalog".

Sesja - 2p.

W projekcie używamy sesji do pobierania informacji o odpowiednim użytkowniku. Listener **AuthenticationApplicationListener.java** przy udanej autentykacji dodaje do sesji atrybut **userId**. Atrybut ten jest wykorzystywany między innymi w widoku **catalogs.html** do pobrania katalogów tego użytkownika, który jest aktualnie zalogowany. Ten atrybut sesji służy nam też do sprawdzania, który użytkownik jest zalogowany i czy ma on dostęp do konkretnych funkcjonalności.

AuthenticationApplicationListener.java:

```
1 package pl.noteally.config;
2
3
4 import jakarta.servlet.http.HttpSession;
5 import lombok.AllArgsConstructor;
6 import org.springframework.context.event.EventListener;
7 import org.springframework.security.authentication.event.InteractiveAuthenticationSuccessEvent;
8 import org.springframework.stereotype.Component;
9 import pl.noteally.data.User;
10
11 @Component
12 @AllArgsConstructor
13 public class AuthenticationApplicationListener {
14
15     private final HttpSession session;
16     @EventListener
17     public void handleInteractiveAuthenticationSuccess(InteractiveAuthenticationSuccessEvent event) {
18         User user = (User) event.getAuthentication().getPrincipal();
19         session.setAttribute("userId", user.getId());
20     }
21 }
```

Fragment kontrolera **CatalogsController.java** pobierający katalogi na podstawie atrybutu sesji **userId**:

```
31 @GetMapping("/")
32 public String getCatalogsByUserId(Model model, HttpSession session, HttpServletRequest request) {
33     Optional<String> sortValue = Arrays.stream(request.getCookies()).filter(
34         c -> c.getName().equals("catalogCookie" + session.getAttribute("userId")))
35         .map(Cookie::getValue).findAny();
36
37     if(sortValue.isEmpty())
38         return "redirect:/login";
39
40     if (sortValue.get().equals("ASC"))
41         return "redirect:/catalogs/ASC";
42     else if (sortValue.get().equals("DESC"))
43         return "redirect:/catalogs/DESC";
44     else if (sortValue.get().equals("notesASC"))
45         return "redirect:/catalogs/notesASC";
46     else if (sortValue.get().equals("notesDESC"))
47         return "redirect:/catalogs/notesDESC";
48
49     List<Catalog> catalogList = catalogService.getCatalogsByUserId((Integer) session.getAttribute("userId"));
50     Optional<User> user = userService.getUserById((Integer) session.getAttribute("userId"));
51     model.addAttribute("catalogs", catalogList);
52     model.addAttribute("user", user.get());
53     return "catalogs";
54 }
```

Ciasteczka - 1p.

Ciasteczka służące do zapamiętywania kryteriów i kierunków sortowania tworzone są przy rejestracji użytkownika.

Kontroler **Log_Reg_Controller**, fragment metody **register()**:

```
82 Cookie catalogCookie = new Cookie("catalogCookie" + user.getId(), "default");
83 catalogCookie.setMaxAge(60 * 60 * 24 * 365 * 10);
84 Cookie noteCookie = new Cookie("noteCookie" + user.getId(), "default");
85 noteCookie.setMaxAge(60 * 60 * 24 * 365 * 10);
86 response.addCookie(catalogCookie);
87 response.addCookie(noteCookie);
88
```

Usługa REST - 3p.

Na potrzeby serwisu Noteally, w oddzielnym projekcie **IsNamePolishREST** została skonfigurowana usługa REST pod adresem **localhost:9090**. Usługa ta jest wykorzystywana przy rejestracji użytkownika i porównuje ona wpisane imię do bazy polskich imion **polishNames.txt**.

Fragment serwisu **IsNamePolishService.java** znajdującego się w projekcie **IsNamePolishREST** jest on odpowiedzialny za porównanie odebranego imienia z plikiem **polishNames.txt**:

```
12  @Service
13  public class IsNamePolishService {
14
15      private List<String> nameList;
16      public IsNamePolishService()
17      {
18          nameList = new ArrayList<>();
19          try{
20              BufferedReader reader = new BufferedReader(new FileReader(new File( pathname: "src/polishNames.txt")));
21              String name;
22              while ((name = reader.readLine()) != null)
23              {
24                  nameList.add(name);
25              }
26          } catch(IOException e) {
27              e.printStackTrace();
28          }
29      }
30      public boolean isNamePolish(String name)
31      {
32          if(name == null) return false;
33          if(nameList.contains(name)) return true;
34          else return false;
35      }
36  }
```

Fragment serwisu **RestNameService** odpowiedzialnego za wysyłanie zapytań do naszego serwera REST:

```
7  @Service
8  public class RestNameService {
9
10     @Autowired
11     private RestTemplate restTemplate;
12
13     public boolean isNamePolish(String name)
14     {
15         Boolean response = restTemplate.getForObject( url: "http://localhost:9090/" + name, Boolean.class);
16         if(response == null) response = false;
17         return response;
18     }
19 }
```

Fragment kontrolera **Log_Reg_Controller** (metoda **register**) odpowiedzialny za sprawdzenie, czy podane imię jest polskie:

```
66         if(!restNameService.isNamePolish(user.getName())) {
67             model.addAttribute( attributeName: "polishNameError", attributeValue: "Name must be Polish");
68             errors = true;
69         }
```

Wygląd - (WCAG 2.1)

Do sprawdzenia poziomu dostępności naszej aplikacji użyto wtyczki do przeglądarki WAVE. Pokazuje ona tylko kilka nieznacznych błędów. Dotyczą one braku tekstu przy linkach służących do sortowania lub tworzenia zasobów. Wyświetlają się one, gdyż postanowiliśmy zastosować ikonki, zamiast tekstu.

